

FINAL PROJECT REPORT: ARTIFICIAL NEURAL NETWORK

Tanggal: 17 Desember 2024

Dibuat oleh: Jevon Al Salgus

Anggota Kelompok:

- Ikhwanul Arif
- Jevon Al Salgus

Disetujui oleh:**Project Leader**

Prof. Trio Adiono, Ph.D

Tanggal**Tanda Tangan****Member**

Nana Sutisna S.T., M.T., Ph.D.

Nur Ahmadi, S.T., M.Eng., Ph.D.

Erwin Setiawan

Michael Jonathan

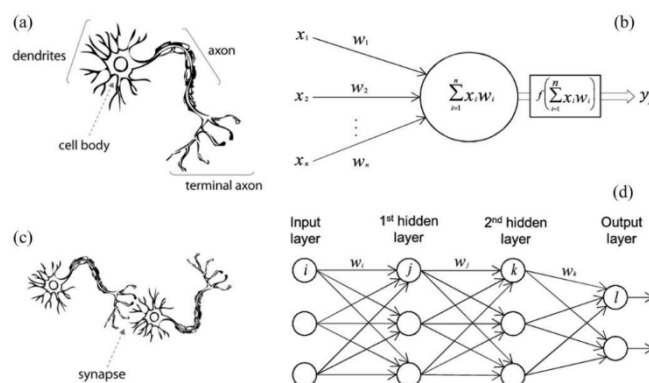
Abdillah Aziz

Tanggal**Tanda Tangan**

PENDAHULUAN

Artificial Neural Network (ANN) atau Jaringan Saraf Tiruan merupakan salah satu pemodelan kompleks yang dapat memprediksi bagaimana ekosistem merespon perubahan variabel lingkungan dengan terinspirasi oleh cara kerja sistem saraf biologis, khususnya pada sel otak manusia dalam memproses informasi. Algoritma yang paling populer digunakan oleh ANN adalah *supervised learning*. Namun, hal tersebut tidak menutup kemungkinan bahwa ANN dapat digunakan untuk algoritma *semi-supervised* dan *unsupervised learning*. Dikarenakan model ANN terinspirasi oleh sistem saraf biologis manusia, arsitekturnya pun dibuat seperti struktur otak manusia dimana terdiri dari neuron yang saling terhubung satu sama lain dan bentuk yang kompleks dan nonlinier.

Neuron di dalam tubuh manusia dibagi menjadi 3 bagian yaitu *dendrites*, *cell body*, dan *axon*. Dimana *dendrites* yaitu sebuah sinyal masukan dan dipengaruhi oleh *weight* (bobot), *cell body* adalah tempat komputasi sinyal yang masuk dan *weight* (bobot) sendiri menghasilkan sinyal *output* yang akan dikirimkan ke neuron lain. Sedangkan *axon* adalah bagian yang mengirimkan sinyal *output* kepada neuron lain yang terhubung dengannya. Berdasarkan penjelasan tersebut ANN dapat direpresentasikan menjadi 3 bagian yang terdiri dari layer masukan (*input*) dan layer keluar (*output*), dan layer tersembunyi yang memproses masukan dari layer masukan menjadi sesuatu yang dapat diterima layer keluaran. Kemudian hasil dari masing-masing layer tersembunyi ini disebut *activation* atau *node value*. Berikut ini adalah penggambaran dari ANN:



Gambar 1 Perbandingan sel saraf dan artificial neuron: (a) sel saraf pada otak manusia; (b) artificial neuron; (c) sinapsis pada otak manusia; (d) sinapsis pada ANN [1]

Contoh penerapan Neural Network:

- Pengenalan Wajah (Facial Recognition)

Pengenalan wajah merupakan penerapan teknologi computer vision untuk mendeteksi dan mengenali wajah manusia. Sistem computer vision melakukan proses ekstraksi, analisis, dan klasifikasi citra untuk memahami informasi secara otomatis. Teknologi ini dapat memanfaatkan berbagai jenis data, termasuk video, gambar tunggal, tampilan dari berbagai sudut pandang fotografi, serta data tiga dimensi. Sebagai contoh, fitur Google Photos menggunakan teknologi ini untuk mengkategorikan foto secara otomatis berdasarkan identifikasi wajah dalam

- Sensor Sidik Jari (Fingerprint Sensor)

Sensor sidik jari pada perangkat smartphone juga merupakan salah satu penerapan *neural networks*. Pada sistem ini, perangkat menyimpan data biometrik pemiliknya dan menjalankan perintah berdasarkan kecocokan data fisik yang diterima saat ada kontak. Pengguna dapat membuka kunci perangkat dengan mudah jika data biometrik yang terdeteksi sesuai. Jika data tidak cocok, sistem dapat mengalihkan ke metode otentikasi lain, seperti sistem kunci alternatif.

- Pengenalan Tulisan Tangan (Handwriting Recognition)

Pengenalan tulisan tangan, atau *handwriting recognition*, adalah teknologi yang memungkinkan komputer untuk menginterpretasikan dan memahami input berupa tulisan tangan dari berbagai sumber, termasuk penggunaan stylus. Teknologi yang mendasari pengenalan tulisan tangan ini adalah *Optical Character Recognition* (OCR), yang memanfaatkan algoritma untuk mengenali teks dari gambar. Beberapa algoritma yang digunakan dalam teknologi ini antara lain *Convolutional Neural Networks* (CNNs), *Support Vector Machines* (SVMs), dan *Hidden Markov Models* (HMMs).

Melalui teknologi ini, gambar yang berisi tulisan tangan dapat diubah menjadi file elektronik, mengurangi kebutuhan penyimpanan fisik dan memungkinkan pengelolaan data yang lebih efisien. Hal ini memberikan keuntungan dalam hal penghematan ruang penyimpanan serta mempermudah proses filtrasi dan manajemen informasi.

- Data Mining untuk Bisnis

Dalam konteks bisnis, *neural networks* dapat digunakan untuk proses *data mining*, yang berfungsi untuk mengidentifikasi pola dalam database yang memungkinkan pemahaman lebih mendalam tentang perilaku pelanggan. Teknologi ini membantu bisnis untuk merancang strategi pemasaran yang lebih efektif dan efisien, serta meningkatkan penjualan dengan biaya yang lebih rendah.

- Prakiraan (Forecasting)

Pada dasarnya, *neural networks* dirancang untuk mengenali pola dan mendeteksi kemungkinan-kemungkinan yang dapat terjadi. Karena kemampuan ini, *neural networks* sangat berguna dalam membuat prakiraan untuk berbagai fenomena, seperti ramalan cuaca, fluktuasi harga saham, dan sebagainya. Hasil dari prediksi ini bahkan dapat mencapai tingkat akurasi yang sangat tinggi, sehingga bermanfaat dalam perencanaan dan pengambilan keputusan yang lebih baik [2].

PEMODELAN ANN

Pemodelan ANN dilakukan berdasarkan studi kasus dengan asumsi-asumsi sebagai berikut:

- Terdapat enam buah jenis makanan khas Indonesia: sate maranggi, soto, karedok, gudeg, ikan bakar, dan rendang.
- Seorang responden mencicipi setiap makanan khas Indonesia tersebut dan menentukan tingkat rasa asam, manis, asin, dan pedas dari masing-masing makanan.
- Setelah menentukan masing-masing tingkat rasa, responden tersebut menentukan makanan yang dia suka. Asumsi nilai supervisor ($[suka, tidak\ suka]$) untuk makanan yang ia suka memiliki nilai $[1,0]$ dan untuk makanan yang ia tidak suka memiliki nilai $[0,1]$. Studi kasus ini dirangkum dalam **Error! Reference source not found..**

Tabel 1 Studi kasus selera makanan khas Indonesia seorang responden

Makanan	Tingkat Rasa				Supervisor	Selera
	Asam	Manis	Asin	Pedas		
Sate Maranggi	2	10	5	3	[1,0]	Suka
Soto	7	2	3	3	[0,1]	Tidak Suka
Karedok	6	8	1	6	[0,1]	Tidak Suka
Gudeg	3	10	3	1	[1,0]	Suka
Ikan Bakar	6	9	5	6	[1,0]	Suka
Rendang	3	2	6	10	[0,1]	Tidak Suka

Input Model ANN

Input	Penjelasan
$K = \begin{bmatrix} 2 & 7 & 6 & 3 & 6 & 3 \\ 10 & 2 & 8 & 10 & 9 & 2 \\ 5 & 3 & 1 & 3 & 5 & 6 \\ 3 & 3 & 6 & 1 & 6 & 10 \end{bmatrix}$	Matriks K adalah matriks input yang mencakup nilai tingkat rasa (asam, manis, asin, pedas) untuk berbagai jenis makanan khas Indonesia. Dalam kasus ini Setiap baris K mewakili jenis makanan tertentu (Sate Maranggi, Soto, dll.). Kolom mencerminkan intensitas rasa untuk atribut tertentu.

$K_p = \begin{bmatrix} 2 & 7 & 6 & 3 & 6 & 3 \\ 10 & 2 & 8 & 10 & 9 & 2 \\ 5 & 3 & 1 & 3 & 5 & 6 \\ 3 & 3 & 6 & 1 & 6 & 10 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$	Matriks K_p adalah versi matriks K yang telah ditambahkan padding di baris terakhir. Padding berupa baris tambahan dengan nilai 111 bertujuan untuk memungkinkan operasi perkalian matriks dengan bias dalam ANN. Mengapa ada padding? Dalam ANN, bias ditambahkan ke setiap node sebagai offset tambahan. Dengan padding, bias dapat direpresentasikan sebagai elemen matriks (dalam satu baris) sehingga perhitungan bias menjadi bagian dari operasi matriks tanpa tambahan perhitungan terpisah.
$WB_2 = \begin{bmatrix} -1.2 & 1.3 & 1.7 & -1.3 & -1.3 \\ 0.3 & 0.5 & 0.2 & 1 & -1 \\ 0.6 & 0.1 & 0.8 & 1.5 & -1 \\ 1.3 & -1.2 & -1.4 & 1.3 & -0.9 \\ 1.3 & 0.3 & 0.5 & 0.4 & -1 \end{bmatrix}$	Matriks WB_2 adalah matriks weight-bias untuk layer tersembunyi (<i>hidden layer</i>) dalam ANN. Elemen-elemen dalam WB_2 mencakup, <i>Weight</i> (pembobotan), representasi kekuatan hubungan antara input tertentu (rasa asam, manis, dll.) dengan neuron di hidden layer.
$WB_3 = \begin{bmatrix} 5.2 & -0.3 & 0.8 & -3.5 & 0.1 & -1.5 \\ -4.8 & 0.1 & 0.7 & 4.0 & 0.9 & -1.4 \end{bmatrix}$	Matriks WB_3 adalah matriks weight-bias untuk layer output dalam ANN. WB_3 terdiri dari weight, pembobotan dari hidden layer ke output layer dan bias, Offset tambahan untuk setiap neuron di output layer.

Output Model ANN

Output Model ANN: Input Hidden Layer (Z₂)

	1	2	3	4	5	6	7
1	13.9000	-5.9000	-4.2000	11.9000	3.9000	-5.1000	
2	8.6000	5.7000	11	6.5000	12.3000	12.1000	
3	9.7000	10.3000	13.2000	5.7000	16.5000	20.8000	
4	-13.4000	5.5000	3.7000	-11.9000	-3.1000	5.2000	
5	8.3000	11.4000	12.1000	7.8000	14.4000	10.5000	
6							

Gambar 2 Input Hidden Layer (Z₂)

Output Model ANN: Activation Function Hidden Layer (A₂)

	1	2	3	4	5	6	7
1	1.0000	0.0027	0.0148	1.0000	0.9802	0.0061	
2	0.9998	0.9967	1.0000	0.9985	1.0000	1.0000	
3	0.9999	1.0000	1.0000	0.9967	1.0000	1.0000	
4	1.5151e-06	0.9959	0.9759	6.7904e-06	0.0431	0.9945	
5	0.9998	1.0000	1.0000	0.9996	1.0000	1.0000	
6							

Gambar 3 Activation Function Hidden Layer (A₂)

Output Model ANN: Input Output Layer (Z₃)

	1	2	3	4	5	6	7
1	4.3000	-4.3706	-4.2387	4.2977	4.0460	-4.3493	
2	-4.5003	4.2702	4.1326	-4.5028	-4.2323	4.2489	
3							

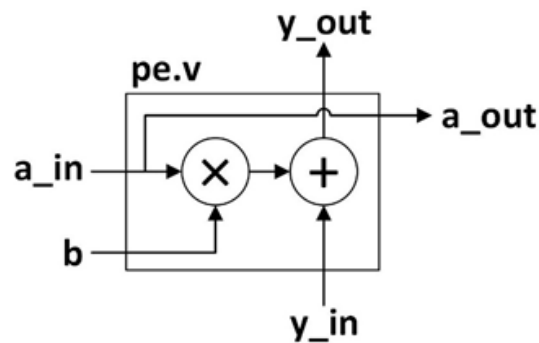
Gambar 4 Input Output Layer (Z₃)

Output Model ANN: Activation Function Output Layer (A₃)

	1	2	3	4	5	6	7
1	0.9866	0.0125	0.0142	0.9866	0.9828	0.0128	
2	0.0110	0.9862	0.9842	0.0110	0.0143	0.9859	
3							

Gambar 5 Activation Function Output Layer (A₃)

PERANCANGAN RTL PROCESSING ELEMENT (PE) UNTUK SYSTOLIC ARRAY



Gambar 6 Modul processing element (pe.v)

Fungsi PE

Processing Element (PE) adalah unit komputasi dasar dalam arsitektur array sistolik. Fungsinya adalah melakukan operasi aritmetika sederhana, seperti perkalian dan penjumlahan (multiplication and addition), yang menjadi dasar untuk proses perkalian matriks pada ANN. Input: PE menerima dua input utama a_{in} dan b , serta nilai y_{in} dari elemen sebelumnya. PE mengalikan a_{in} dengan b , kemudian menjumlahkan hasilnya dengan y_{in} , menghasilkan output y_{out} .

Ekspektasi output, output y_{out} adalah hasil akumulasi operasi dalam array sistolik, yang merupakan bagian dari hasil akhir perkalian matriks. Dan nilai a_{out} menjadi input a_{in} ke PE berikutnya dalam array sistolik.

Fixed Point Q5.10

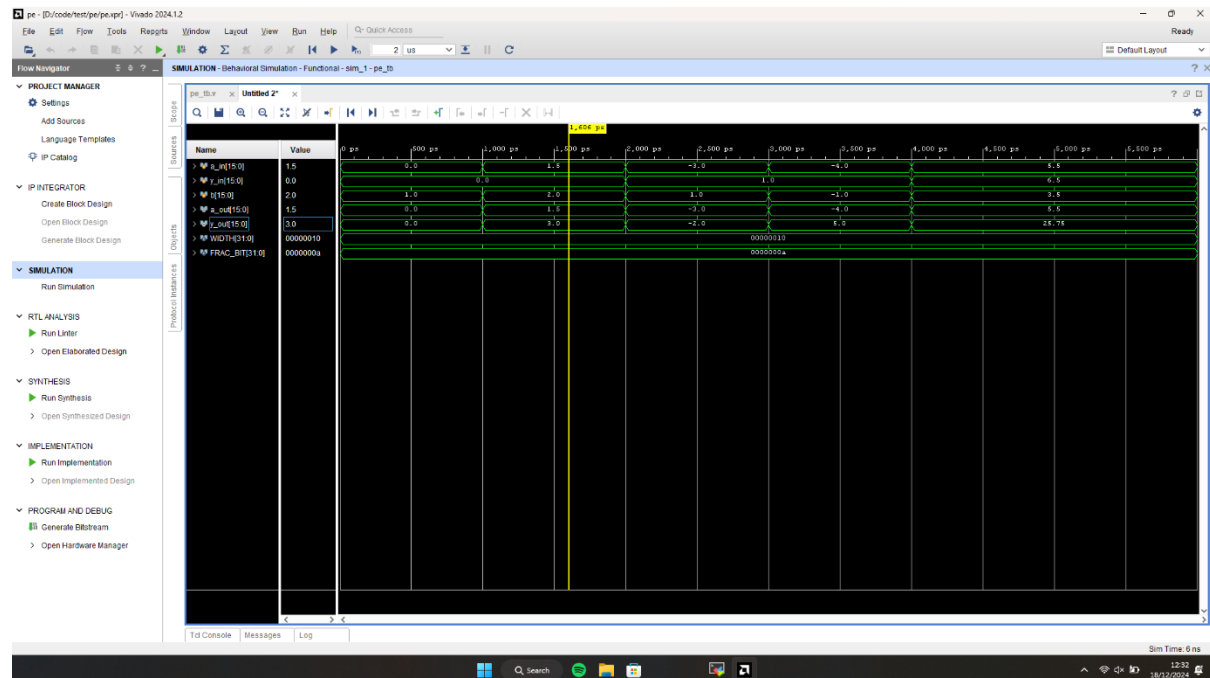
Fixed-point Q5.10 adalah format representasi bilangan biner yang terdiri dari 1 bit signed, 5bit bagian integer, dan 10bit bagian fraksional. Format ini digunakan untuk menyimpan bilangan desimal dalam bentuk biner dengan presisi tinggi dan efisiensi, khususnya dalam perangkat keras seperti ANN systolic. Bit signed menunjukkan apakah bilangan tersebut positif atau negatif. Jika bit signed adalah 0, bilangan positif, sedangkan jika bit signed adalah 1, bilangan negatif

Lima bit setelah bit signed merepresentasikan bagian integer, yaitu nilai sebelum titik biner, sedangkan sepuluh bit terakhir merepresentasikan bagian fraksional, yaitu nilai

setelah titik biner. Bagian fraksional dihitung dengan menjumlahkan kontribusi setiap bit sesuai dengan bobotnya, mulai dari 2^{-1} hingga 2^{-10} . Misalnya, pada bilangan Q5.10 "0000010101100000", bit signed adalah 0 sehingga bilangan ini positif. Lima bit berikutnya 00001 merepresentasikan nilai integer sebesar 1, sedangkan sepuluh bit terakhir 0101100000 merepresentasikan bagian fraksional sebesar 0.34375, yang dihitung dari $2^{-2} + 2^{-4} + 2^{-5}$. Dengan menggabungkan nilai integer dan fraksional, bilangan ini direpresentasikan dalam desimal sebagai 1.34375.

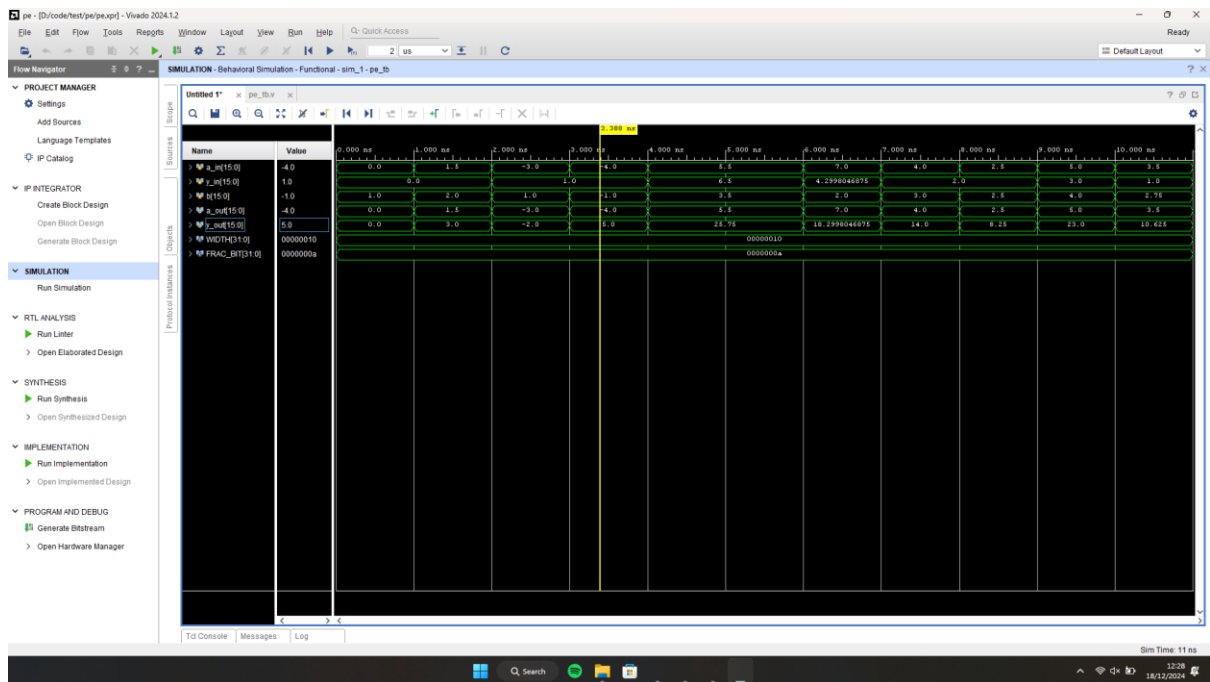
Format fixed-point Q5.10 sangat cocok digunakan dalam ANN systolic karena memberikan keseimbangan antara presisi yang memadai dan efisiensi perangkat keras. Dengan menggunakan fixed-point, operasi aritmetika seperti perkalian dan penjumlahan dapat dilakukan dengan lebih cepat dibandingkan floating-point. Selain itu, Q5.10 memberikan rentang nilai yang cukup luas, dari -16.0 hingga +15.999, sehingga dapat menangani berbagai perhitungan numerik dalam propagasi jaringan neural.

Hasil Pengujian Modul RTL PE dengan Testbench yang Diberikan



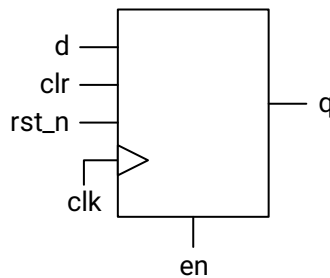
Gambar 7 Hasil Pengujian Modul RTL PE dengan Testbench yang Diberikan

Hasil Pengujian PE dengan Minimal Lima Buah *Custom Test Vector*



Gambar 8 Hasil Pengujian PE dengan Minimal Lima Buah *Custom Test Vector*

PERANCANGAN REGISTER UNTUK SYSTOLIC ARRAY



Gambar 9 Modul register (register.v)

Fungsi Register

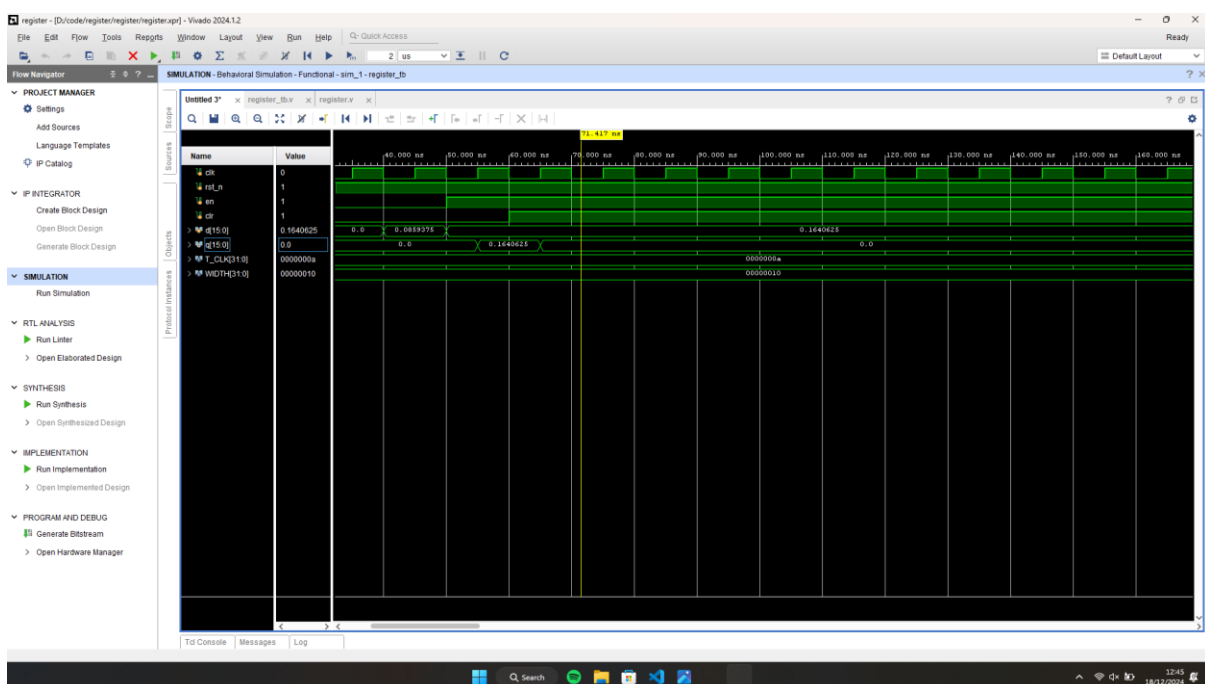
Register dalam array sistolik digunakan untuk menyimpan sementara nilai input dan output selama proses komputasi berlangsung. Fungsinya termasuk:

1. Menyediakan sinkronisasi data di antara elemen array.
2. Menyimpan data sementara sebelum diteruskan ke modul berikutnya.

Ekspektasi Output:

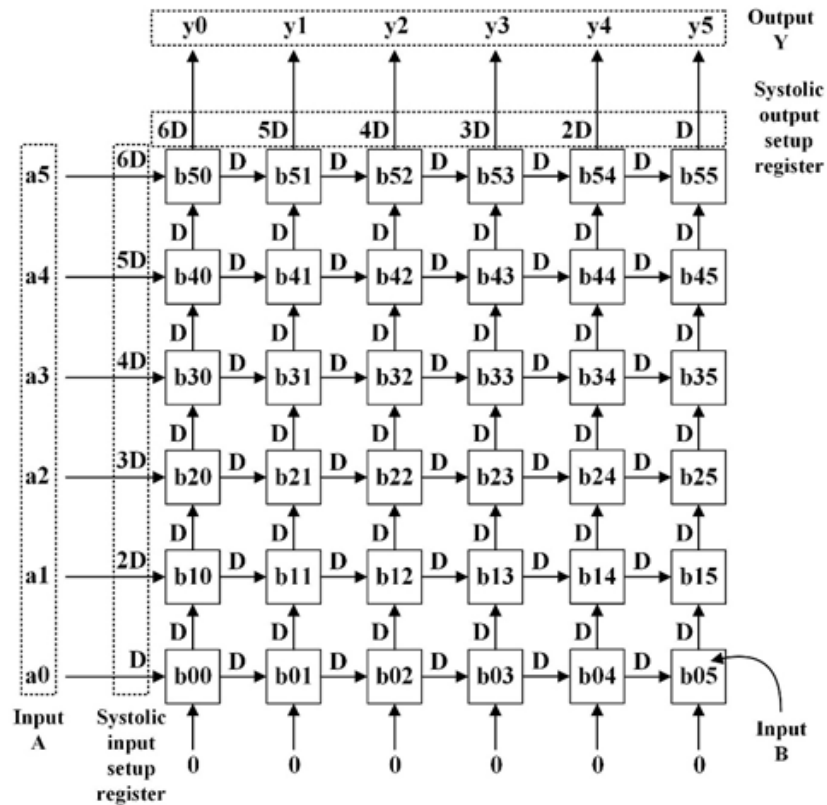
1. Pada setiap clock cycle, nilai register q mencerminkan nilai input d, selama sinyal enable (en) aktif.

Hasil Pengujian Model RTL Register dengan Testbench



Gambar 10 Hasil Pengujian Model RTL Register dengan Testbench

PERANCANGAN SYSTOLIC ARRAY 6x6



Gambar 11 Modul matriks array sistolik 6x6

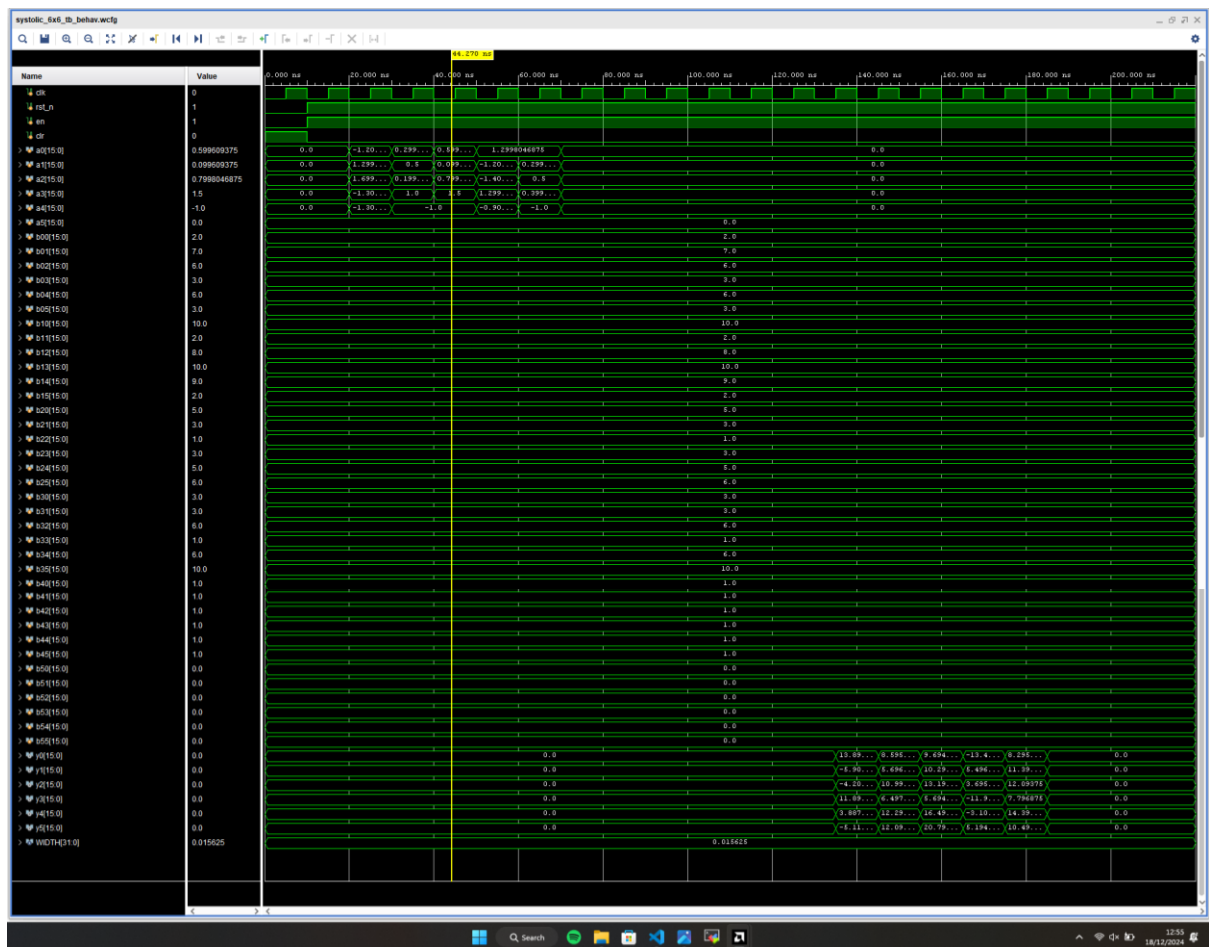
Fungsi Systolic Array 6x6

Systolic array adalah arsitektur perangkat keras yang digunakan untuk melakukan operasi matriks secara paralel dan efisien. Dalam konteks ANN, systolic array sering digunakan untuk perkalian matriks, yang merupakan operasi penting dalam propagasi data melalui layer-layer jaringan.

Fungsi Systolic Array 6x6:

1. **Perkalian Matriks:** Systolic array 6x6 dirancang untuk menghitung hasil perkalian dua matriks berukuran 6×6 . Setiap elemen pada matriks hasil dihitung menggunakan elemen-elemen dari matriks input (a_0 - a_5 dan b_{00} - b_{55}) melalui operasi berbasis Processing Element (PE).

Perbandingan Hasil Keluaran Simulasi RTL Array Systolic dengan Output Model



Gambar 12 Hasil Keluaran Simulasi RTL Array Systolic

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	13.9000	-5.9000	-4.2000	11.9000	3.9000	-5.1000								
2	8.6000	5.7000	11	6.5000	12.3000	12.1000								
3	9.7000	10.3000	13.2000	5.7000	16.5000	20.8000								
4	-13.4000	5.5000	3.7000	-11.9000	-3.1000	5.2000								
5	8.3000	11.4000	12.1000	7.8000	14.4000	10.5000								
6														
7														
8														
9														
10														
11														
12														
13														
14														
15														
16														

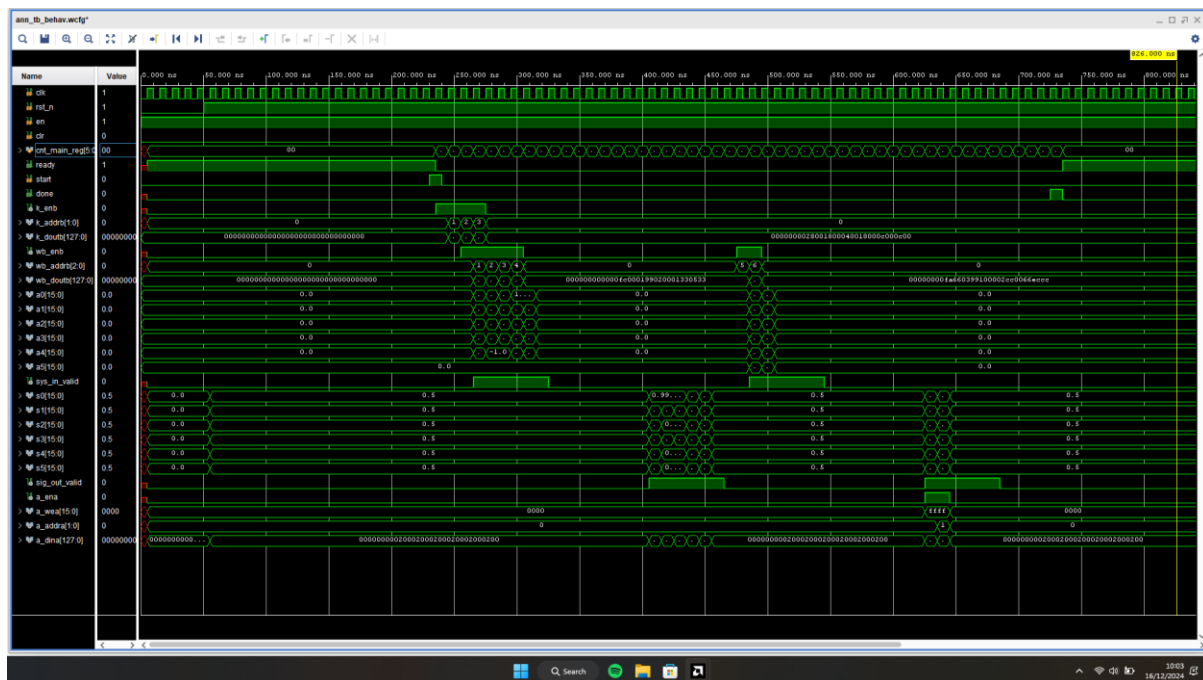
Gambar 13 Hasil Output Model

VERIFIKASI ANN CORE DAN AXIS ANN

Tahapan yang dilakukan dalam proses verifikasi ini ada beberapa Langkah, yaitu:

1. Tahapan yang pertama adalah menjalankan simulasi dari Source File yang sudah disediakan, kemudian membandingkan hasilnya dengan Model Solusi yang sudah dibuat sebelumnya untuk memastikan bahwa keluaran yang diberikan oleh modul ANN sudah benar.
2. Tahapan yang kedua adalah melanjutkan untuk simulasi Axis_ANN untuk memastikan bahwa data yang masuk dan keluar sudah sesuai dengan sinyal control yang diberikan.
3. Tahapan berikutnya adalah melakukan penyesuaian dan simulasi dengan menggunakan Source File yang sudah dibuat, untuk menunjukkan apakah Source File yang dibuat sudah sesuai atau belum. Untuk verifikasi dilakukan dengan cara membandingkan dengan hasil simulasi dari Source File yang disediakan. Pada Langkah ini terdapat beberapa kendala, yaitu output kedua yang ditampilkan tidak sesuai dengan yang seharusnya, dan keluaran output muncul lebih cepat satu clock dibandingkan dengan Source File yang disediakan. Permasalahan tersebut dapat diatasi dengan cara menambahkan 6 buah register, yaitu 1 register pada masing masing keluaran systolic 6x6.

Hasil Verifikasi Menggunakan Source Files Solusi



Gambar 14 Hasil Verifikasi Menggunakan Source Files Solusi

penyesuaian ini, hasil sigmoid output 2 pada waveform tb_ann menjadi konsisten dengan nilai A3. Langkah ini berhasil memperbaiki sinkronisasi antara sistem dan meningkatkan keakuratan hasil verifikasi ANN Core.

INTEGRASI DESAIN SOC ANN

Hasil Synthesis

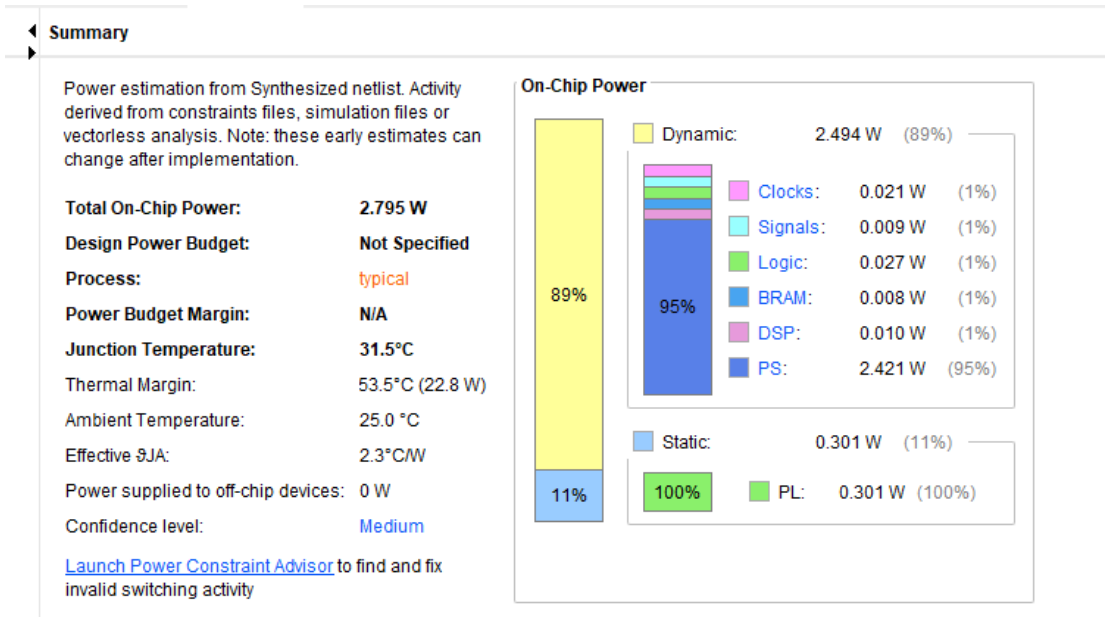
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.537 ns	Worst Hold Slack (WHS): -0.090 ns	Worst Pulse Width Slack (WPWS): 3.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): -247.907 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 7266	Number of Failing Endpoints: 0
Total Number of Endpoints: 30787	Total Number of Endpoints: 30787	Total Number of Endpoints: 10019
Timing constraints are not met.		

Gambar 16 Hasil Synthesis SOC ANN

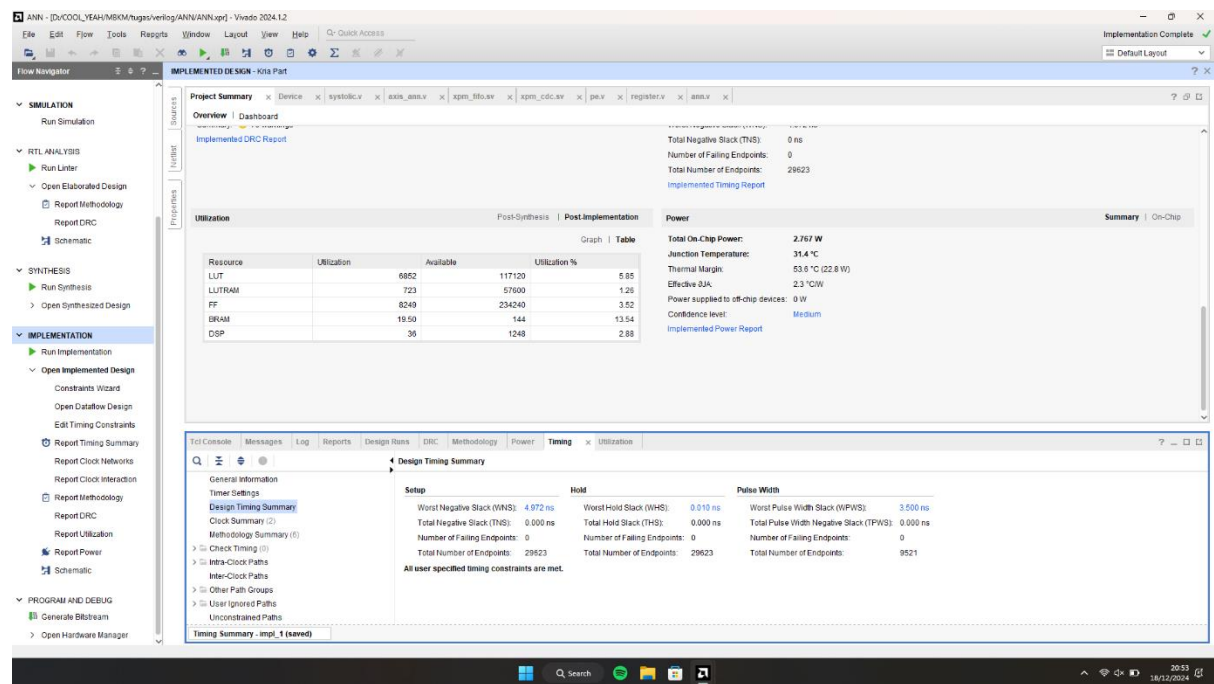
Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	F8 Muxes (29280)	Block RAM Tile (144)	DSPs (1248)	BUFG_PS (96)	PS8 (1)
design_1_wrapper	7642	8743	110	64	30	20	36	2	1
design_1_i (design_1)	7642	8743	110	64	30	20	36	2	1
axi_dma_0 (design_1_axi_dma_0_0)	2342	3603	30	0	0	5	0	0	0
axi_interconnect_0 (design_1_axi_interconnect_0_0)	2752	2938	16	4	0	0	0	0	0
axis_ann_0 (design_1_axis_ann_0_0)	1465	1139	64	60	30	15	36	0	0
proc_sys_reset_0 (design_1_proc_sys_reset_0_0)	19	40	0	0	0	0	0	0	0
smartconnect_0 (design_1_smartconnect_0_0)	351	480	0	0	0	0	0	0	0
smartconnect_1 (design_1_smartconnect_1_0)	449	543	0	0	0	0	0	0	0
zynq_ultra_ps_e_0 (design_1_zynq_ultra_ps_e_0_0)	264	0	0	0	0	0	0	2	1

Gambar 17Utilization Report



Gambar 18 Power Report

Hasil Implementasi



Gambar 19 Hasil Implementasi SOC ANN

Perbandingan antara Hasil Integrasi SoC dan Output Model

Prose Software

Z3 (Software):

```
[ [ 4.29997127 -4.3705759 -4.23872749 4.29768248 4.04595627 -4.34928807 ]
  [-4.50027444 4.27023894 4.13256859 -4.50279334 -4.23233852 4.24894242 ] ]
```

A3 (Software):

```
[ [ 0.9866127 0.01248608 0.01422079 0.98658244 0.98280777 0.01275131 ]
  [ 0.01098396 0.98621426 0.98421165 0.01095663 0.01431063 0.9859217 ] ]
```

Lama eksekusi software: 0.001850 detik

```
////////////////////////////////////
////////////////////////////////////
```

Prose Hardware

Mengirim data ke FPGA

Data transfer selesai!

Data transfer selesai!

Output Buffer:

Address 00: 0x703f0

Address 01: 0x3ef0002

Address 02: 0x803fe

Address 03: 0x0

Address 04: 0x3f3000a

Address 05: 0x903f9

Address 06: 0x3f50003

Matrix Hardware Result(A3):

```
[ [ 0.984375 0.00683594 0.00195312 0.98339844 0.99804688 0.0078125 ]
  [ 0.00976562 0.98730469 0.99316406 0.00878906 0.00292969 0.98925781 ] ]
```

Matrix Hardware Result Rounded:

```
[ [ 1. 0. 0. 1. 1. 0. ]
  [ 0. 1. 1. 0. 0. 1. ] ]
```

Lama eksekusi hardware: 0.001268 detik

Gambar 20 Hasil Uji Dengan Script Python

```

Command Window
New to MATLAB? See resources for Getting Started.
>> MatrixA2
Z2 =
    13.9000    -5.9000    -4.2000    11.9000     3.9000    -5.1000
     8.6000     5.7000    11.0000     6.5000    12.3000    12.1000
     9.7000    10.3000    13.2000     5.7000    16.5000    20.8000
    -13.4000     5.5000     3.7000    -11.9000    -3.1000     5.2000
     8.3000    11.4000    12.1000     7.8000    14.4000    10.5000

A2 expanded =
    1.0000    0.0027    0.0148    1.0000    0.9802    0.0061
    0.9998    0.9967    1.0000    0.9985    1.0000    1.0000
    0.9999    1.0000    1.0000    0.9967    1.0000    1.0000
    0.0000    0.9959    0.9759    0.0000    0.0431    0.9945
    0.9998    1.0000    1.0000    0.9996    1.0000    1.0000
    1.0000    1.0000    1.0000    1.0000    1.0000    1.0000

Z3 =
     4.3000    -4.3706    -4.2387     4.2977     4.0460    -4.3493
    -4.5003     4.2702     4.1326    -4.5028    -4.2323     4.2489

A3 =
    0.9866    0.0125    0.0142    0.9866    0.9828    0.0128
    0.0110    0.9862    0.9842    0.0110    0.0143    0.9859

```

Gambar 21 Hasil Simulasi Model

Hasil integrasi SoC yang diuji menggunakan script python pada jupyter notebook sudah sesuai dengan hasil simulasi model yang sudah dibuat pada matlab setelah dibandingkan. Dapat dilihat pada gambar 20, matrix hardware result (A3) jika dibandingkan dengan hasil A3 pada matlab memang tidak sama persis, tetapi setelah dibulatkan (*Resul Rounded*) makan nilainya akan menjadi 1 atau 0, jadi jika berbeda 0.005 saja, maka tidak mempengaruhi secara signifikan

Tabel 2 Hasil Uji Dengan Script Python

Makanan	Supervisor	Prediksi ANN
Sate Maranggi	[1, 0]	[1, 0] ✓
Soto	[0, 1]	[0, 1] ✓
Karedok	[0, 1]	[0, 1] ✓
Gudeg	[1, 0]	[1, 0] ✓
Ikan Bakar	[1, 0]	[1, 0] ✓
Rendang	[0, 1]	[0, 1] ✓

Hasil prediksi dari ANN sepenuhnya sesuai dengan kolom supervisor dalam tabel studi kasus. Dengan kata lain, ANN berhasil memprediksi preferensi responden

("Suka" atau "Tidak Suka") untuk setiap makanan khas Indonesia sesuai dengan data kasus.

Kendala yang Dihadapi pada Proses Integrasi Desain SoC

Kendala yang didapati saat proses Integrasi Desain SOC terdapat pada proses setelah membuat Block Design, dimana untuk Alamat dan besar memori pada Address Editor yang salah sehingga mengakibatkan gagal dalam megekseskui Generate Bitstream, hal ini dapat diatasi dengan melakukan unsign dan sign ulang dari alamat memori yang akan digunakan, selain itu juga perlu untuk menyesuaikan parameter yang digunakan pada desain yang dibuat dengan design yang diberikan.

Perbandingan Waktu Komputasi ANN di Hardware dengan Waktu Komputasi pada Software Python

1. Inisialisasi FPGA dan Buffer:

```
from pynq import Overlay

ol = Overlay("./design_1.bit")

from pynq import allocate

import numpy as np
import time

input_size = 44
inputbuffer = allocate(shape=(input_size,), dtype=np.uint32)

output_size = 8
outputbuffer = allocate(shape=(output_size,), dtype=np.uint32)
```

Gambar 22 Inisialisasi FPGA dan Buffer

- File bitstream (design_1.bit) diunggah ke FPGA Kria KV melalui kabel Ethernet.
- Buffer input dan output dialokasikan menggunakan library PYNQ. Ukuran buffer disesuaikan dengan data ANN, yakni 44 untuk input dan 8 untuk output.

2. Eksekusi di Hardware (FPGA):

```

inputbuffer[0] = 0x0533f833
inputbuffer[1] = 0xfacc06cc
inputbuffer[2] = 0x0000facc
inputbuffer[3] = 0x00000000
inputbuffer[4] = 0x02000133
inputbuffer[5] = 0x040000cc
inputbuffer[6] = 0x0000fc00
inputbuffer[7] = 0x00000000
inputbuffer[8] = 0x0060266
inputbuffer[9] = 0x6000333
inputbuffer[10] = 0x0000fc00
inputbuffer[11] = 0x00000000
inputbuffer[12] = 0xf8330533
inputbuffer[13] = 0x0533fa66
inputbuffer[14] = 0x0000fc66
inputbuffer[15] = 0x00000000
inputbuffer[16] = 0x01330533
inputbuffer[17] = 0x01990200
inputbuffer[18] = 0x0000fc00
inputbuffer[19] = 0x00000000

inputbuffer[20] = 0xfeccl4cc
inputbuffer[21] = 0xf2000333
inputbuffer[22] = 0xfa000066
inputbuffer[23] = 0x00000000
inputbuffer[24] = 0x0060eccc
inputbuffer[25] = 0x100002cc
inputbuffer[26] = 0xfa660399
inputbuffer[27] = 0x00000000

inputbuffer[28] = 0x1c000800
inputbuffer[29] = 0x0c001800
inputbuffer[30] = 0x0c001800
inputbuffer[31] = 0x00000000
inputbuffer[32] = 0x08002800
inputbuffer[33] = 0x28002000
inputbuffer[34] = 0x08002400
inputbuffer[35] = 0x00000000
inputbuffer[36] = 0x0c001400
inputbuffer[37] = 0x0c000400
inputbuffer[38] = 0x18001400
inputbuffer[39] = 0x00000000
inputbuffer[40] = 0x0c00c000
inputbuffer[41] = 0x04001800
inputbuffer[42] = 0x28001800
inputbuffer[43] = 0x00000000

```

Gambar 23 InputBuffer

```

in_buff = []
for i in range(input_size):
    in_buff.append(hex(inputbuffer[i]))

```

```
print(in_buff)
```

```

['0x533fb33', '0xfacc06cc', '0xfacc', '0x0', '0x2000133', '0x40000cc', '0xfc00', '0x0', '0x660266', '0x6000333',
'0xfc00', '0x0', '0xfb330533', '0x533fa66', '0xfc66', '0x0', '0x1330533', '0x1990200', '0xfc00', '0x0', '0xfeccl4
cc', '0xf2000333', '0xfa000066', '0x0', '0x66eccc', '0x100002cc', '0xfa660399', '0x0', '0x1c000800', '0xc001800',
'0xc001800', '0x0', '0x8002800', '0x28002000', '0x8002400', '0x0', '0xc001400', '0xc000400', '0x18001400', '0x0',
'0xc00c00', '0x4001800', '0x28001800', '0x0']

```

```
len(inputbuffer)
```

```
44
```

```

start_hw = time.time()
dma = ol.axi_dma_0
dma.sendchannel.transfer(inputbuffer)
dma.recvchannel.transfer(outputbuffer)
end_hw = time.time()
print(f'Hardware exec time: {(end_hw - start_hw)*1000}')

```

```
Hardware exec time: 4.354715347290039
```

```
pred_hw = np.zeros((2,6))
```

```

pred_hw[0][0] = ((outputbuffer[0] & 0x000ffff) / 1024.0)
pred_hw[0][1] = (((outputbuffer[0] & 0xffff000) >> 16) / 1024.0)
pred_hw[0][2] = ((outputbuffer[1] & 0x000ffff) / 1024.0)
pred_hw[0][3] = (((outputbuffer[1] & 0xffff000) >> 16) / 1024.0)
pred_hw[0][4] = ((outputbuffer[2] & 0x000ffff) / 1024.0)
pred_hw[0][5] = (((outputbuffer[2] & 0xffff000) >> 16) / 1024.0)

```

```

pred_hw[1][0] = ((outputbuffer[4] & 0x000ffff) / 1024.0)
pred_hw[1][1] = (((outputbuffer[4] & 0xffff000) >> 16) / 1024.0)
pred_hw[1][2] = ((outputbuffer[5] & 0x000ffff) / 1024.0)
pred_hw[1][3] = (((outputbuffer[5] & 0xffff000) >> 16) / 1024.0)
pred_hw[1][4] = ((outputbuffer[6] & 0x000ffff) / 1024.0)
pred_hw[1][5] = (((outputbuffer[6] & 0xffff000) >> 16) / 1024.0)

```

```
np.round(pred_hw)
```

```

array([[1., 0., 0., 1., 1., 0.],
      [0., 1., 1., 0., 0., 1.]])

```

Gambar 24 Eksekusi Hardware

- Data input (inputbuffer) diisi dengan nilai 32bit yang disiapkan untuk ANN.
- Data ditransfer ke FPGA melalui DMA (Direct Memory Access), yang mengelola transfer input ke hardware dan pengambilan output dari hardware.
- Waktu eksekusi dihitung menggunakan Python time module.

3. Eksekusi di Software (Python Numpy):

```
# Definisikan matriks WB2
WB2 = np.array([[[-1.2, 1.3, 1.7, -1.3, -1.3],
                 [0.3, 0.5, 0.2, 1, -1],
                 [0.6, 0.1, 0.8, 1.5, -1],
                 [1.3, -1.2, -1.4, 1.3, -0.9],
                 [1.3, 0.3, 0.5, 0.4, -1]])

# Definisikan matriks WB3
WB3 = np.array([[5.2, -0.3, 0.8, -3.5, 0.1, -1.5],
                [-4.8, 0.1, 0.7, 4.0, 0.9, -1.4]])

# Definisikan matriks Kp
Kp = np.array([[2, 7, 6, 3, 6, 3],
               [10, 2, 8, 10, 9, 2],
               [5, 3, 1, 3, 5, 6],
               [3, 3, 6, 1, 6, 10],
               [1, 1, 1, 1, 1, 1]])

start_sw = time.time()

for i in range(10):
    A = WB2 @ Kp
    A_sig = 1 / (1+np.exp(-A))
    A_sig_padded = np.append(A_sig, np.ones((1, A_sig.shape[1])), axis=0)
    Z = WB3 @ A_sig_padded
    Z_sig = 1/(1+np.exp(-Z))

end_sw = time.time()
sw1 = end_sw - start_sw
print(f'Software execution time: {(end_sw-start_sw)} detik')

Software execution time: 0.0028471946716308594 detik
```

Gambar 25 Eksekusi Software

- Perhitungan ANN dilakukan menggunakan operasi matriks dengan numpy. Matriks bobot dan bias (WB2, WB3) dikalikan dengan input, diikuti oleh penerapan fungsi aktivasi sigmoid.
- Iterasi komputasi dijalankan hingga 10^6 kali, sama seperti hardware, untuk mendapatkan waktu total.

4. Iterasi:

```
start_hw = time.time()
dma = ol.axi_dma_0
for i in range(10):
    dma.sendchannel.transfer(inputbuffer)
    dma.recvchannel.transfer(outputbuffer)
end_hw = time.time()
hw1 = end_hw - start_hw
print(f'Hardware exec time: {(end_hw - start_hw)} detik')
```

Hardware exec time: 0.0042493343353271484 detik

```
start_hw = time.time()
dma = ol.axi_dma_0
for i in range(100):
    dma.sendchannel.transfer(inputbuffer)
    dma.recvchannel.transfer(outputbuffer)
end_hw = time.time()
hw2 = end_hw - start_hw
print(f'Hardware exec time: {(end_hw - start_hw)} detik')
```

Hardware exec time: 0.018998146057128906 detik

```
start_hw = time.time()
dma = ol.axi_dma_0
for i in range(1000):
    dma.sendchannel.transfer(inputbuffer)
    dma.recvchannel.transfer(outputbuffer)
end_hw = time.time()
hw3 = end_hw - start_hw
print(f'Hardware exec time: {(end_hw - start_hw)} detik')
```

Hardware exec time: 0.1759319305419922 detik

```
start_hw = time.time()
dma = ol.axi_dma_0
for i in range(10000):
    dma.sendchannel.transfer(inputbuffer)
    dma.recvchannel.transfer(outputbuffer)
end_hw = time.time()
hw4 = end_hw - start_hw
print(f'Hardware exec time: {(end_hw - start_hw)} detik')
```

Hardware exec time: 1.7020559310913086 detik

```
start_hw = time.time()
dma = ol.axi_dma_0
for i in range(100000):
    dma.sendchannel.transfer(inputbuffer)
    dma.recvchannel.transfer(outputbuffer)
end_hw = time.time()
hw5 = end_hw - start_hw
print(f'Hardware exec time: {(end_hw - start_hw)} detik')
```

Hardware exec time: 16.830425024032593 detik

```
start_hw = time.time()
dma = ol.axi_dma_0
for i in range(1000000):
    dma.sendchannel.transfer(inputbuffer)
    dma.recvchannel.transfer(outputbuffer)
end_hw = time.time()
hw6 = end_hw - start_hw
print(f'Hardware exec time: {(end_hw - start_hw)} detik')
```

Hardware exec time: 167.86325192451477 detik

Gambar 26 Iterasi Hardware


```

start_sw = time.time()

for i in range (10000):
    A = WB2 @ Kp
    A_sig = 1 / (1+np.exp(-A))
    A_sig_padded = np.append(A_sig, np.ones((1, A_sig.shape[1])), axis=0)
    Z = WB3 @ A_sig_padded
    Z_sig = 1/(1+np.exp(-Z))

end_sw = time.time()
sw4 = end_sw - start_sw
print(f'Software execution time: {(end_sw-start_sw)} detik')

```

Software execution time: 1.7668931484222412 detik

```

start_sw = time.time()

for i in range (100000):
    A = WB2 @ Kp
    A_sig = 1 / (1+np.exp(-A))
    A_sig_padded = np.append(A_sig, np.ones((1, A_sig.shape[1])), axis=0)
    Z = WB3 @ A_sig_padded
    Z_sig = 1/(1+np.exp(-Z))

end_sw = time.time()
sw5 = end_sw - start_sw
print(f'Software execution time: {(end_sw-start_sw)} detik')

```

Software execution time: 17.29870629310608 detik

```

start_sw = time.time()

for i in range (1000000):
    A = WB2 @ Kp
    A_sig = 1 / (1+np.exp(-A))
    A_sig_padded = np.append(A_sig, np.ones((1, A_sig.shape[1])), axis=0)
    Z = WB3 @ A_sig_padded
    Z_sig = 1/(1+np.exp(-Z))

end_sw = time.time()
sw6 = end_sw - start_sw
print(f'Software execution time: {(end_sw-start_sw)} detik')

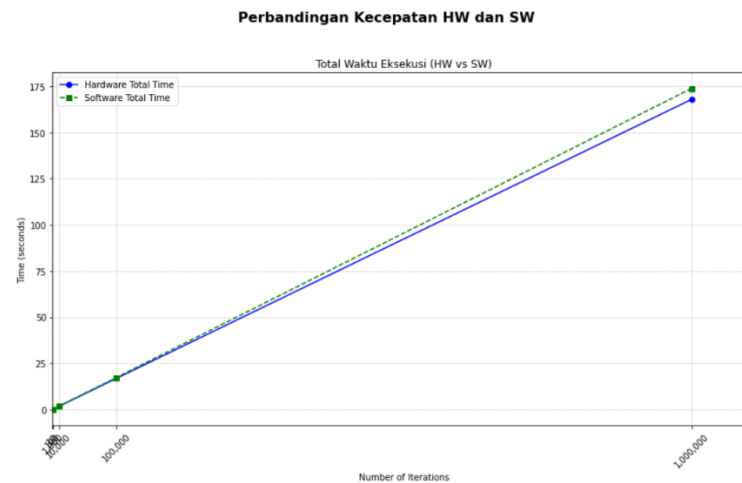
```

Software execution time: 173.7911765575409 detik

Gambar 27 Iterasi Software

- Perhitungan ANN dilakukan menggunakan operasi matriks dengan numpy. Matriks bobot dan bias (WB2, WB3) dikalikan dengan input, diikuti oleh penerapan fungsi aktivasi sigmoid.
- Iterasi komputasi dijalankan hingga 10^6 kali, sama seperti hardware, untuk mendapatkan waktu total.

5. Analisis dan Visualisasi:



Gambar 28 Visualisasi dan Tabel Iterasi

Berdasarkan hasil yang diperoleh, dapat disimpulkan bahwa hardware menunjukkan performa yang lebih baik dibandingkan software pada iterasi besar, sedangkan software hanya unggul pada iterasi kecil, yaitu pada jumlah iterasi 10. Pada iterasi kecil, overhead pengiriman data ke hardware menjadi salah satu penyebab software terlihat lebih cepat. Namun, ketika jumlah iterasi bertambah, keunggulan hardware semakin terlihat karena proses paralel yang dilakukan oleh arsitektur systolic array dalam menjalankan Artificial Neural Network (ANN). Hal ini memungkinkan waktu per iterasi hardware tetap cenderung konstan meskipun jumlah data meningkat, sedangkan software yang menjalankan proses secara sekuensial di CPU menunjukkan peningkatan waktu eksekusi per iterasi.

Jika bias pada hardware dioptimalisasi, maka performa dapat ditingkatkan dengan cara memperbaiki mekanisme pengiriman data dan mengurangi overhead pada proses transfer data antara CPU dan hardware (misalnya FPGA). Selain itu, pengoptimalan algoritme ANN pada arsitektur systolic array juga dapat dilakukan untuk memaksimalkan efisiensi komputasi paralel. Dengan mengurangi bias ini, performa hardware dapat semakin diunggulkan bahkan pada iterasi kecil, sehingga memberikan kecepatan yang lebih konsisten pada berbagai skala data. Hal ini menegaskan bahwa dengan optimalisasi bias dan overhead, hardware memiliki potensi untuk unggul secara menyeluruh dibandingkan software dalam memproses data.

REFERENSI

- [1] N. Anisa, "Mengenal Artificial Neural Network," BINUS University, 14 February 2022. [Online]. Available: <https://sis.binus.ac.id/2022/02/14/mengenal-artificial-neural-network/>. [Accessed 18 December 2024].
- [2] Admin, "Apa itu Neural Network? Pengertian, Sejarah, Konsep dan Contohnya," codingstudio, 13 July 2023. [Online]. Available: <https://codingstudio.id/blog/apa-itu-neural-network/>. [Accessed 18 December 2024].